SoftBook — Register / Seed New Company

Example company_id: 10004
Admin login: Admin / Admin@123
Tenant DB: softaccdb_1_3107
Master DB: companymaster (from appsettings DefaultConnection)

Step	Database	What it does
1	companymaster	Insert company + connection string (routing)
2	softaccdb_1_3107	Ensure ledger settings mapping columns
3	softaccdb_1_3107	Default accounts (tab_party_master) + tab_ledger_settings
4	softaccdb_1_3107	Fiscal year + Admin user + Admin role
5	softaccdb_1_3107	Full Admin permissions on shared tabs menu

> **Not seeded (by design):** tab_account_group_master and tab_voucher_type_master — existing live company (e.g. 10001) does not rely on these for normal app use.

> **Run order:** Step 1 on *companymaster*, then Steps 2–5 on tenant DB *softaccdb_1_3107*. Change company_name / Host / Password if your environment differs.

STEP 1 — CompanyMaster (routing)

Database: companymaster

```
-- ============================================================
-- STEP 1 ONLY â€” Register company in CompanyMaster
-- Database: companymaster
-- (appsettings.json â†’ ConnectionStrings:DefaultConnection)
-- ============================================================
-- After this, seed tenant DB softaccdb_1_3107 using:
--   Scripts/Register_Company_10004_Tenant.sql
-- Full guide:
--   Docs/Register_New_Company_Guide.md
-- ============================================================

INSERT INTO tab_company_master (
    company_id,
    company_name,
    connection_string,
    is_enabled,
    priority
)
VALUES (
    10004,
    'Hetak Demo Co 10004',   -- <<< change display name as needed
    'Host=192.168.1.207;Database=softaccdb_1_3107;Username=postgres;Password=Soft123',
    TRUE,
    0
)
ON CONFLICT (company_id) DO UPDATE
SET
    company_name      = EXCLUDED.company_name,
    connection_string = EXCLUDED.connection_string,
```

```
    is_enabled        = TRUE;

-- Verify routing
SELECT company_id, company_name, connection_string, is_enabled, priority, created_at
FROM tab_company_master
WHERE company_id = 10004;
```

STEPS 2–5 — Tenant DB seed

Database: softaccdb_1_3107

```
-- ============================================================
-- Tenant seed for company_id = 10004
-- Database: softaccdb_1_3107  (must match tab_company_master.connection_string)
-- Prerequisites:
--   1) Run Scripts/Register_Company_10004_CompanyMaster.sql on companymaster
--   2) Tenant schema + shared tabs menu already exist on this DB
--
-- NOTE: Does NOT seed tab_account_group_master or tab_voucher_type_master.
--       Existing live company (e.g. 10001) does not rely on those for app use.
--       Only seeds: default accounts â†’ ledger settings â†’ FY â†’ Admin â†’ permissions.
-- ============================================================

-- ------------------------------------------------------------
-- STEP 2: Ledger mapping columns
-- ------------------------------------------------------------
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS sales_revenue_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS purchase_expense_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS cgst_output_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS sgst_output_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS igst_output_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS cgst_input_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS sgst_input_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS igst_input_account_id INT;
ALTER TABLE tab_ledger_settings ADD COLUMN IF NOT EXISTS round_off_account_id INT;

-- ------------------------------------------------------------
-- STEP 3: Default accounts (tab_party_master) + ledger settings
-- mapping: company_name = account name, party_type = account type, display_name = code
-- ------------------------------------------------------------
DO $$
DECLARE
    v_company_id   INT := 10004;
    v_acc_sales    INT;
    v_acc_pur      INT;
    v_acc_cgst_in  INT;
    v_acc_sgst_in  INT;
    v_acc_igst_in  INT;
    v_acc_cgst_out INT;
    v_acc_sgst_out INT;
    v_acc_igst_out INT;
    v_acc_round    INT;
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM tab_party_master
        WHERE company_id = v_company_id
          AND company_name = 'Sales Account'
          AND party_type = 'Sales'
    ) THEN
        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'Sales Account', 'Sales', 'ACC001')
        RETURNING party_id INTO v_acc_sales;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'Purchase Account', 'Purchase', 'ACC002')
```

```
        RETURNING party_id INTO v_acc_pur;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'CGST Input', 'Tax', 'TAX001')
        RETURNING party_id INTO v_acc_cgst_in;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'SGST Input', 'Tax', 'TAX002')
        RETURNING party_id INTO v_acc_sgst_in;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'IGST Input', 'Tax', 'TAX003')
        RETURNING party_id INTO v_acc_igst_in;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'CGST Output', 'Tax', 'TAX004')
        RETURNING party_id INTO v_acc_cgst_out;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'SGST Output', 'Tax', 'TAX005')
        RETURNING party_id INTO v_acc_sgst_out;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'IGST Output', 'Tax', 'TAX006')
        RETURNING party_id INTO v_acc_igst_out;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'Round Off', 'Other', 'ACC003')
        RETURNING party_id INTO v_acc_round;

        INSERT INTO tab_party_master (company_id, company_name, party_type, display_name)
        VALUES (v_company_id, 'Cash Account', 'Cash', 'CASH01');

        IF EXISTS (SELECT 1 FROM tab_ledger_settings WHERE company_id = v_company_id) THEN
            UPDATE tab_ledger_settings SET
                sales_revenue_account_id     = v_acc_sales,
                purchase_expense_account_id  = v_acc_pur,
                cgst_output_account_id       = v_acc_cgst_out,
                sgst_output_account_id       = v_acc_sgst_out,
                igst_output_account_id       = v_acc_igst_out,
                cgst_input_account_id        = v_acc_cgst_in,
                sgst_input_account_id        = v_acc_sgst_in,
                igst_input_account_id        = v_acc_igst_in,
                round_off_account_id         = v_acc_round
            WHERE company_id = v_company_id;
        ELSE
            INSERT INTO tab_ledger_settings (
                company_id,
                sales_revenue_account_id, purchase_expense_account_id,
                cgst_output_account_id, sgst_output_account_id, igst_output_account_id,
                cgst_input_account_id, sgst_input_account_id, igst_input_account_id,
                round_off_account_id
            ) VALUES (
                v_company_id,
                v_acc_sales, v_acc_pur,
                v_acc_cgst_out, v_acc_sgst_out, v_acc_igst_out,
                v_acc_cgst_in, v_acc_sgst_in, v_acc_igst_in,
                v_acc_round
            );
        END IF;
    END IF;
END $$;

SELECT * FROM tab_ledger_settings WHERE company_id = 10004;

SELECT party_id, company_name, party_type, display_name
FROM tab_party_master
WHERE company_id = 10004
```

```
  AND company_name IN (
      'Sales Account','Purchase Account','CGST Input','SGST Input','IGST Input',
      'CGST Output','SGST Output','IGST Output','Round Off','Cash Account'
  );

-- ------------------------------------------------------------
-- STEP 4: Fiscal year + Admin user/role
-- Password: Admin@123  (SHA256 Base64 â€” PasswordHelper)
-- ------------------------------------------------------------
DO $$
DECLARE
    v_company_id INT := 10004;
    v_role_id    INT;
    v_user_id    INT;
    v_pwd_hash   TEXT := '6G94qKPK8LYNjnTllCqm2G3BUM08AzOK7yW30tfjrMc=';
BEGIN
    UPDATE tab_fiscal_year_master SET is_current = FALSE WHERE company_id = v_company_id;

    INSERT INTO tab_fiscal_year_master (company_id, start_date, end_date, is_current, fiscal_year_label)
    SELECT v_company_id, DATE '2026-04-01', DATE '2027-03-31', TRUE, '2026-27'
    WHERE NOT EXISTS (
        SELECT 1 FROM tab_fiscal_year_master
        WHERE company_id = v_company_id
          AND start_date = DATE '2026-04-01'
          AND end_date   = DATE '2027-03-31'
    );

    UPDATE tab_fiscal_year_master
    SET is_current = TRUE, fiscal_year_label = COALESCE(fiscal_year_label, '2026-27')
    WHERE company_id = v_company_id
      AND start_date = DATE '2026-04-01'
      AND end_date   = DATE '2027-03-31';

    SELECT role_id INTO v_role_id
    FROM tab_role_master
    WHERE company_id = v_company_id AND LOWER(role_name) = 'admin'
    LIMIT 1;

    IF v_role_id IS NULL THEN
        INSERT INTO tab_role_master (company_id, role_name, is_active, is_deleted)
        VALUES (v_company_id, 'Admin', TRUE, FALSE)
        RETURNING role_id INTO v_role_id;
    END IF;

    SELECT user_id INTO v_user_id
    FROM tab_user_master
    WHERE company_id = v_company_id AND LOWER(username) = 'admin'
    LIMIT 1;

    IF v_user_id IS NULL THEN
        INSERT INTO tab_user_master (company_id, username, password_hash, full_name, email, phone, is_active)
        VALUES (v_company_id, 'Admin', v_pwd_hash, 'Administrator', 'admin@company.com', NULL, TRUE)
        RETURNING user_id INTO v_user_id;
    ELSE
        UPDATE tab_user_master
        SET password_hash = v_pwd_hash, is_active = TRUE
        WHERE user_id = v_user_id;
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM tab_user_role
        WHERE company_id = v_company_id AND user_id = v_user_id AND role_id = v_role_id
    ) THEN
        INSERT INTO tab_user_role (user_id, role_id, company_id)
        VALUES (v_user_id, v_role_id, v_company_id);
    END IF;

    RAISE NOTICE 'Seeded company_id=%, user_id=%, role_id=%', v_company_id, v_user_id, v_role_id;
```

```
END $$;

SELECT * FROM tab_fiscal_year_master WHERE company_id = 10004;
SELECT user_id, username, email, is_active FROM tab_user_master WHERE company_id = 10004;
SELECT * FROM tab_role_master WHERE company_id = 10004;
SELECT * FROM tab_user_role WHERE company_id = 10004;

-- ------------------------------------------------------------
-- STEP 5: Admin full permissions on shared tabs menu
-- ------------------------------------------------------------
DO $$
DECLARE
    v_company_id INT := 10004;
    v_role_id    INT;
    v_count      INT;
BEGIN
    SELECT role_id INTO v_role_id
    FROM tab_role_master
    WHERE company_id = v_company_id
      AND LOWER(role_name) = 'admin'
      AND COALESCE(is_deleted, FALSE) = FALSE
    LIMIT 1;

    IF v_role_id IS NULL THEN
        RAISE EXCEPTION 'Admin role not found for company_id=% . Run Step 4 first.', v_company_id;
    END IF;

    INSERT INTO tab_role_permission (
        company_id, role_id, tab_id,
        can_view, can_add, can_edit, can_delete, is_active
    )
    SELECT
        v_company_id, v_role_id, t.tab_id,
        TRUE, TRUE, TRUE, TRUE, TRUE
    FROM tabs t
    WHERE COALESCE(t.is_active, TRUE) = TRUE
      AND NOT EXISTS (
          SELECT 1 FROM tab_role_permission p
          WHERE p.company_id = v_company_id
            AND p.role_id = v_role_id
            AND p.tab_id = t.tab_id
      );

    GET DIAGNOSTICS v_count = ROW_COUNT;

    UPDATE tab_role_permission
    SET can_view = TRUE, can_add = TRUE, can_edit = TRUE, can_delete = TRUE, is_active = TRUE
    WHERE company_id = v_company_id AND role_id = v_role_id;

    RAISE NOTICE 'Admin role_id=% for company_id=% â€” inserted % new tab permissions',
        v_role_id, v_company_id, v_count;
END $$;

SELECT COUNT(*) AS admin_tab_perms
FROM tab_role_permission
WHERE company_id = 10004
  AND role_id = (
      SELECT role_id FROM tab_role_master
      WHERE company_id = 10004 AND LOWER(role_name) = 'admin' LIMIT 1
  );

SELECT COUNT(*) AS total_active_tabs
FROM tabs
WHERE COALESCE(is_active, TRUE) = TRUE;
```